

# Sharpchess structuur template in Arc42

SharpChess is een full-stack schaakwebapplicatie waarmee gebruikers zich kunnen registreren, inloggen en online kunnen schaken via een strikt gescheiden frontend, backend en data/infrastructuuropzet. Dit Word-document spiegelt de hoofdstukindeling van de DokChess-architectuurwebsite, terwijl de inhoud is herschreven voor het SharpChess-project.

Technologiefocus React + TypeScript, ASP.NET, EF-Core en JWT

## Architectuuroverzicht

SharpChess is een webapplicatie voor schaken. Het systeem bestaat uit een React-frontend, een ASP.NET Core-backend en een databaselaag op basis van EF Core. Via de browser maken gebruikers een account aan, bevestigen ze hun e-mailadres, loggen ze in en spelen ze schaakpartijen. De backend verwerkt de spelregels en de spelstatus. Dit document beschrijft de architectuur en de ontwerpbeslissingen van het project. Het bevat de systeemeisen, de grenzen tussen de componenten, de interacties tijdens runtime en de gemaakte technische keuzes. Het is geschreven voor docenten, medestudenten en beheerders die de opbouw van het systeem willen inzien.

- Inleiding en Doelstellingen Dit hoofdstuk beschrijft SharpChess en de eisen waaraan de architectuur voldoet.

## 1.1 Vereistenoverzicht

SharpChess heeft een client-serverarchitectuur en bestaat uit drie lagen. De frontend is de interface. De backend bevat de logica en de beveiliging. Het dataproject slaat de gegevens op via

## Entity Framework Core.

Overzicht van kernvereisten

## Gebied Beschrijving

**Gebruikersbeheer** Gebruikers moeten zich kunnen registreren, hun e-mailadres kunnen bevestigen en veilig kunnen inloggen. **Spelinteractie** Gebruikers moeten een schaakbord kunnen bekijken, zetten kunnen indienen en actuele spelstatusinformatie kunnen ontvangen. **Regelhandhaving** Schaakzetten moeten in de backend worden gevalideerd, zodat de browser nooit bepaalt of een zet geldig is. **Persistentie** Relevante gegevens zoals accounts, tokens en partijen moeten consistent worden opgeslagen via de backend. **Architectuur** De oplossing moet onderhoudbaar blijven door frontend-, backend- en data/infrastructuurverantwoordelijkheden te scheiden.

## 1.2 Kwaliteitsdoelen

Naast de functionele eisen heeft SharpChess specifieke kwaliteitsdoelen. Deze doelen bepalen de technologiekeuze, de projectstructuur en de verdeling van verantwoordelijkheden. Prioritering van kwaliteitsdoelen

## Prioriteit Kwaliteitsdoel Reden

### Onderhoudbaarheid

Het project moet begrijpelijk en aanpasbaar blijven tijdens ontwikkeling en beoordeling.

### Correctheid

Schaakregels, authenticatiestromen en toestandsovergangen moeten consistent werken.

### Beveiliging

Wachtwoorden, tokens en beveiligde endpoints moeten veilig worden behandeld.

Scheiding van verantwoordelijkheden Elk project moet een duidelijke verantwoordelijkheid hebben om architectuurvervuiling te voorkomen.

### Testbaarheid

Belangrijke logica moet verifieerbaar zijn via geautomatiseerde tests en API-tests.

### Bruikbaarheid

De interface moet kernacties zoals registratie, inloggen en zetten indienen eenvoudig maken.

## 1.3 Stakeholders

De volgende stakeholders beïnvloeden de inhoud en focus van de architectuur. Stakeholders en belangen Stakeholder Belang in het systeem Eindgebruikers Willen een duidelijke en betrouwbare manier om online te schaken. Studentontwikkelaar Heeft een schone architectuur nodig die realistisch te bouwen, te verklaren en uit te breiden is. Docenten en beoordelaars Hebben inzicht nodig in architectuurredenering, scheiding van verantwoordelijkheden en technische onderbouwing. Toekomstige beheerders Hebben een systeem nodig dat gemakkelijk te begrijpen, debuggen en door te ontwikkelen is zonder kernregels te breken.

## 2. Beperkingen

De scope van SharpChess wordt bepaald door een aantal vaste beperkingen.

### 2.1 Technisch

Technische beperkingen Beperking Impact op de architectuur React + TypeScript frontend De client is geïmplementeerd als een aparte webapplicatie en communiceert uitsluitend via

## HTTP.

ASP.NET Core Web API backend Bedrijfslogica en API-endpoints zijn gecentraliseerd in een toegewijd backendproject. Entity Framework Core voor persistentie Databasetoegang wordt uitsluitend afgehandeld via EF Core in de data/infrastructuurlaag. JWT-gebaseerde authenticatie Beveiligde endpoints zijn afhankelijk van op tokens gebaseerde autorisatie in plaats van serversessies. Aparte projecten Frontend, backend en data/infrastructuur blijven gescheiden om architectuurgrenzen te bewaken.

## 2.2 Organisatorisch

- Het project wordt ontwikkeld in een onderwijscontext en vereist daarom verklaarbare ontwerpbeslissingen.
- De beschikbare ontwikkeltijd is beperkt, waardoor de scope realistisch moet blijven.
- De documentatie is geschreven voor docenten en medestudenten, niet alleen voor eindgebruikers.
- Het systeem moet zo zijn opgezet dat het demonstraties, testen en incrementele oplevering ondersteunt.

## 2.3 Conventies

- Front-end en backend communiceren uitsluitend via REST/HTTP met JSON-payloads.
- De front-end heeft geen directe toegang tot de database.
- De backend geeft nooit EF Core-entiteiten direct terug; antwoorden worden gemapt naar DTO's.
- Entity-Framework Core is de enige route van de backend naar de database.
- C4-diagrammen en een arc42-achtige documentstructuur worden gebruikt om architectuurbeslissingen toe te lichten.

## 3. Context

Contextweergaven beschrijven hoe SharpChess omgaat met externe actoren en aangrenzende systemen.

### 3.1 Bedrijfscontext

Gebruikers spelen online schaak via de browser. SharpChess gebruikt externe diensten voor e-mailverificatie en aanvullende schaakfuncties.

## Figuur 3-1. Systeemcontextdiagram.

Gebruikers communiceren met de front-end. De back-end valideert de schaakzetten en gebruikt een externe e-mailservice voor accountverificatie.

## 3.2 Implementatiecontext

De applicatie kan lokaal via containers draaien, of afzonderlijk in een cloudomgeving worden uitgerold.

## Figuur 3-2. Container/implementatiecontext.

## 4. Oplossingsstrategie

Dit hoofdstuk beschrijft de belangrijkste ontwerpbeslissingen van SharpChess.

### 4.1 Inleiding

SharpChess heeft een klassieke web-architectuur met een strikte scheiding van verantwoordelijkheden.

### 4.2 Structuur

Het systeem bestaat uit drie delen. De frontend is de gebruikersinterface. De back-end biedt de API en valideert de acties en spelregels. De data laag beheert de database en externe koppelingen.

- Frontend: React + TypeScript single-page applicatie.
- Backend: ASP.NET Core Web API met applicatie- en domeinlogica.
- Data/Infrastructuur: EF Core-configuratie, repositories, databasemigraties en externe service-implementaties.

### 4.3 Spelstrategie

De back-end bevat de schaaklogica. De front-end toont de huidige spelstatus. De server valideert alle zetten en slaat de resultaten op. Dit voorkomt dubbele code en zorgt dat de server de enige bron van waarheid is.

### 4.4 Verbinding

De front-end en backend communiceren via REST en JSON. Het systeem gebruikt JWT's voor authenticatie. De back-end benadert de database via EF Core en het repository-patroon.

## 12

## 5. Bouwblokweergave

Dit overzicht toont de bouwblokken van SharpChess, van het grote geheel tot in de detailcomponenten.

## 5.1 Niveau 1

SharpChess bestaat uit de frontend, de backend-API, de data laag en externe diensten.

Figuur 5-1. Containeroverzicht gebruikt als Niveau 1 bouwblokperspectief.

## 5.2 Front-end

De front-end toont de interface, zoals het schaakbord en de formulieren. Het stuurt acties naar de back-end en toont de resultaten.

5.3 Backend-API De backend-API heeft endpoints voor registratie, e-mailverificatie, inloggen en spelacties. Controllers sturen de verzoeken door naar services of handlers.

## 5.4 Applicatielaag

De applicatie-laag voert de acties uit, zoals het registreren en het verwerken van zetten. Deze laag valideert verzoeken, roept de domeinlogica aan en vertaalt DTO's naar domeinobjecten.

## 5.5 Infrastructuurlaag

De infrastructuurlaag bevat op EF Core gebaseerde persistentie, migraties en concrete integraties zoals e-mailbezorging. Het is verantwoordelijk voor technische implementatiedetails die de domein- of presentatielagen niet mogen vervuilen.

5.6 Niveau 2: Backend-API De backend bestaat uit modules voor authenticatie en spelbeheer.

Figuur 5-2. Componentendiagram uit het originele SharpChess-document.

## 5.7 Authenticatiemodule

- Registratiestroom voor het aanmaken van nieuwe accounts.
- E-mailverificatiestroom met eenmalige tokens en verloopafhandeling.
- Inlogstroom die tokens uitgeeft uitsluitend aan geldige en geverifieerde gebruikers.
- Wachtwoordverwerking op basis van veilige hashing in plaats van opslag als platte tekst.

## 5.8 Spelmodule

- Bordtoestandsophaling voor de frontend.
- Zetindiening en zetvalidatie.
- Speltoestandsvoortgang en resultaatafhandeling.
- Opslag van relevante partijeninformatie via backend-persistentieabstracties.

## 6. Runtime weergave

De runtimeweergave legt uit hoe de belangrijkste bouwblokken samenwerken bij belangrijke scenario's.

## 6.1 Registratiestroom

Bij registratie stuurt de frontend de ingevulde gegevens naar de backend. De backend valideert het verzoek, heeft het wachtwoord, slaat de gebruiker op en verstuurt een verificatie-e-mail. De frontend toont daarna het resultaat.

### Figuur 6-1. Registratiesequentiediagram.

## 7. Implementatieweergave

De implementatieweergave beschrijft hoe SharpChess kan worden uitgevoerd tijdens ontwikkeling en in een gehoste omgeving.

7.1 Ontwikkel- en Runtime-infrastructuur Lokaal draait SharpChess in Docker-containers of afzonderlijke processen voor de frontend, backend en een PostgreSQL-database. In productie is de frontend een statische webapplicatie, de backend een webservice en PostgreSQL een beheerde database.

Figuur 7-1. Infrastructuur/implementatiediagram uit het originele SharpChess-document.

CI/CD-pipelines kunnen de backend onafhankelijk van de frontend bouwen en testen. Dit ondersteunt het architectuurdoel dat elk onderdeel onafhankelijk inzetbaar en begrijpelijk moet blijven.

## 8. Concepten

Dit hoofdstuk beschrijft de technische concepten die voor het hele systeem gelden.

### 8.1 Afhankelijkheden

#### Kernafhankelijkheden

Afhankelijkheid Doel in SharpChess ASP.NET Core HTTP-API, routing, dependency injection en autorisatie. Entity Framework Core Databasetoegang, mappings, migraties en persistentie-implementatie. JWT-ondersteuning Op tokens gebaseerde authenticatie voor beveiligde endpoints. FluentValidation Validatie van verzoekmodellen en commando's. Serilog Gestructureerde logging voor diagnostiek en monitoring. React + TypeScript Getypeerde frontend-implementatie voor browserinteractie.

### 8.2 Domeinmodel

Het domeinmodel bestaat uit gebruikers, accounts, partijen, bordposities, zetten en verificatietokens. Dit model staat los van DTO's en de database.

## 8.3 Gebruikersinterface

De frontend is een single-page applicatie (SPA) voor registratie, inloggen en schaken. Het schaakbord wordt bijgewerkt op basis van de antwoorden van de backend.

## 8.4 Validatie

Validatie vindt plaats op meerdere niveaus. Invoermodellen worden gevalideerd op volledigheid en opmaak, terwijl diepere bedrijfsvalidatie controleert of gevraagde acties zinvol zijn binnen de huidige domeinstatus. Dit voorkomt dat controllers en UI-componenten regellogica dragen die in de backend thuishoort.

## 8.5 Foutafhandeling

SharpChess gebruikt foutantwoorden. De API zet validatie- en domeinfouten om in een standaardformaat, en vangt onverwachte fouten centraal af.

## 8.6 Logging

SharpChess gebruikt logging om authenticatie, spelacties en fouten te traceren.

## 8.7 Testbaarheid

Door de strikte scheiding van componenten is het systeem goed testbaar. Domein- en applicatielogica worden gevalideerd met unit tests, en de endpoints met API-tests.

# 9. Ontwerpbeslissingen

Dit hoofdstuk beschrijft de architectuurbeslissingen.

## 9.1 Frontend-connectiviteit

De frontend en backend communiceren via REST. De backend beheert de autorisatie en spelregels centraal. Beide lagen worden afzonderlijk ontwikkeld, getest en uitgerold.

9.2 Positie- en DTO-objecten SharpChess gebruikt aparte objecten voor het domein en voor datatransport. De backend gebruikt het domeinmodel voor de schaaklogica, en DTO's voor de API-communicatie met de frontend.

# 10. Kwaliteitsvereisten

Dit hoofdstuk beschrijft de kwaliteitsvereisten.

## 10.1 Utility Tree

Utility tree voor SharpChess

# Categorie Sub-kwaliteit Belang Toelichting

Onderhoudbaarheid Duidelijke gelaagdheid Hoog Verantwoordelijkheden moeten gemakkelijk te lokaliseren en te wijzigen zijn. Correctheid Regelhandhaving Hoog Illegale zetten en ongeldige stromen moeten betrouwbaar worden geblokkeerd. Beveiliging Authenticatieveiligheid Hoog Inloggegevens en beveiligde bronnen mogen niet onzorgvuldig worden blootgesteld. Testbaarheid Geïsoleerde logica Gemiddeld Kernlogica moet testbaar zijn zonder de UI. Bruikbaarheid Duidelijke feedback Gemiddeld Gebruikers moeten begrijpen wat het systeem doet en waarom acties slagen of mislukken.

## 10.2 Kwaliteitsscenario's

- Wanneer een niet-geverifieerde gebruiker probeert in te loggen, moet de backend het verzoek afwijzen met een duidelijk antwoord dat de status uitlegt.
- Wanneer de frontend een illegale zet stuurt, moet de backend deze afwijzen zonder de opgeslagen spelstatus te beschadigen.
- Wanneer een toekomstige ontwikkelaar de registratielogica moet wijzigen, moet de wijziging mogelijk zijn zonder tegelijkertijd frontend-rendercode of persistentiedetails op dezelfde plek aan te passen.
- Wanneer een onverwachte uitzondering optreedt, moet het systeem voldoende context loggen voor diagnose, terwijl een veilig antwoord wordt teruggestuurd naar de client.

## 11. Risico's

Elke architectuurbeslissing brengt risico's met zich mee. De volgende kwesties zijn bijzonder relevant voor SharpChess.

### 11.1 Frontend-verbinding

SharpChess gebruikt expliciete DTO's en API-tests om de contracten tussen frontend en backend te borgen.

11.2 Inspanning en Scope Een schaakplatform kan gemakkelijk in scope groeien, zeker wanneer functies zoals matchmaking, persistentie, timers en geavanceerde gameplay worden toegevoegd. Het project vereist daarom bewuste scopebeheersing zodat architectuurkwaliteit niet wordt opgeofferd voor het aantal functies.

11.3 Regelcorrectheid en Beveiliging De backend beheert de schaaklogica en authenticatie centraal. Strikte validatie en duidelijke autorisatie borgen de correctheid en veiligheid van het systeem.

## 23

## 12. Woordenlijst

De woordenlijst verzamelt termen die herhaaldelijk in het document worden gebruikt.

## 12.1 Inleiding

Het doel van de woordenlijst is om de architectuurtaal consistent te houden. Dit is met name nuttig in een project dat webarchitectuurtermen combineert met schaakdomeintterminologie.

## 12.2 Termen

### Woordenlijst

**Term** Betekenis in SharpChess  
**DTO** Een data transfer object dat wordt gebruikt voor communicatie tussen frontend en backend.  
**EF Core** Entity Framework Core, gebruikt voor persistentie en databasetoegang in de infrastructuurlaag.  
**JWT** JSON Web Token gebruikt voor stateless authenticatie en autorisatie.  
**Zetvalidatie** De backend-controle die bepaalt of een gevraagde schaakzet legaal is.  
**Spelstatus** De huidige toestand van een schaakpartij, inclusief bordgegevens en beurtvolgorde.  
**Verificatietoken** Een eenmalig token dat wordt gebruikt om een nieuw aangemaakt gebruikersaccount te bevestigen.